

CI/CD

Blue-Green Deployment Architecture & Operations

A complete, inside-out reference for the container build, the GitHub Actions pipeline, the zero-downtime blue-green release mechanism, secrets, migrations, networking & TLS — and the real problems solved along the way.

NestJS 11 · Node 22

Docker · Compose

GitHub Actions → GHCR

Blue-Green · Caddy

PostgreSQL 18 · Redis 8

Single-host · AWS EC2

Repository

github.com/rashedulhasannifty/clickup-sync

Scope

main → automated deploy

Revision

v1.0 · engineering reference

What's in this document

Read top-to-bottom for the full story, or jump to a section. Every section explains **what** the piece does, **how** it works, and **why** it was built this way.

- 01 **Executive Summary** — the system in one page
- 02 **System Overview & Topology** — every container and how they connect
- 03 **One Image, Two Roles** — the web/worker split and why blue-green forced it
- 04 **The Docker Image** — multi-stage build, baked-in dashboard, Prisma
- 05 **The CI/CD Pipeline** — quality → e2e → build → deploy, stage by stage
- 06 **Blue-Green Deployment** — the cutover mechanism in detail
- 07 **Secrets Management** — GitHub Secrets → rendered .env
- 08 **Database Migrations** — one-shot, expand/contract, forward-only
- 09 **Networking & TLS** — Caddy, Cloudflare, certificates
- 10 **Operations Runbook** — deploy, inspect, roll back, DB access
- 11 **Troubleshooting — War Stories** — the real bugs and their fixes
- 12 **Server Provisioning** — readying a fresh host
- 13 **Security Posture & Appendix** — guarantees, file & secret reference, glossary

Executive Summary

Every push to `main` ships a new, tested version of the platform to production with **zero downtime** and an instant rollback path — with no human touching the server.

The platform is a **NestJS** backend that syncs ClickUp data into PostgreSQL and serves a React dashboard. It runs as a set of Docker containers on a **single Linux host**. Deployment is fully automated by **GitHub Actions**: when code lands on `main`, the pipeline runs quality and end-to-end tests, builds a versioned container image, publishes it to the GitHub Container Registry (GHCN), then performs a **blue-green** release on the server over SSH.

Blue-green means two copies of the web application exist — `blue` and `green`. At any moment one is **live** and the other is **idle**. A deploy upgrades the idle copy, health-checks it privately, and only then flips traffic to it. The previous version stays running as a warm rollback target. Users never see a restart.

0s

USER-VISIBLE DOWNTIME PER DEPLOY

1

IMAGE, TWO RUN MODES (WEB / WORKER)

4

PIPELINE STAGES, FULLY AUTOMATED

The five moving parts

Part	Responsibility
GitHub Actions	Runs tests, builds the image, orchestrates the release on <code>push → main</code> .
GHCN (registry)	Stores each build as an immutable, SHA-tagged artifact.
Docker Compose	Defines and runs the container stack on the host (web x2, worker, Postgres, Redis, Caddy).
Caddy	Reverse proxy + automatic HTTPS. Routes traffic to whichever colour is live.
deploy.sh	Host-side script that performs the health-gated blue-green cutover.

DESIGN GOAL

Ship often and safely. The pipeline makes releases boring: they are tested before build, health-gated before traffic moves, and reversible in seconds if something is wrong.

System Overview & Topology

Everything runs as containers on one host, isolated on a private Docker network. Only Caddy is exposed to the internet.

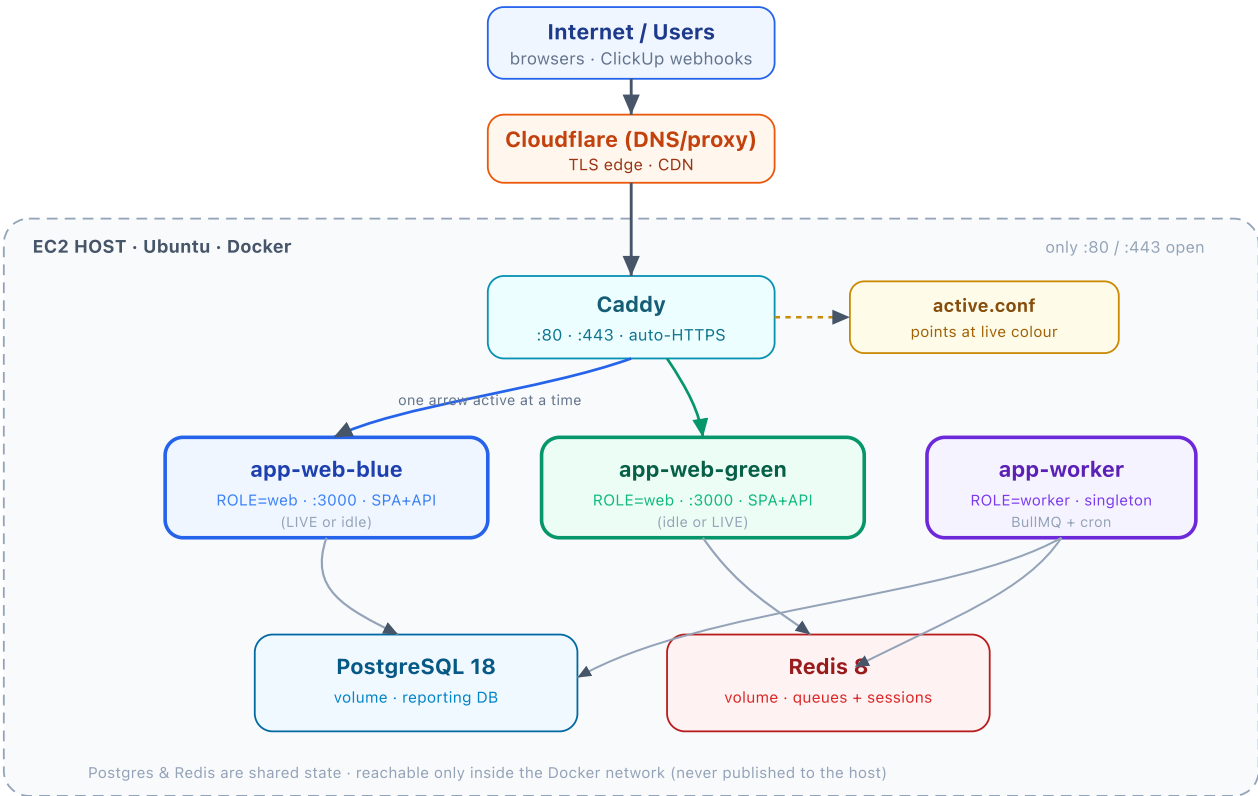


Figure 1 — Production topology. Caddy is the only internet-facing service; it forwards to whichever web colour `active.conf` names. The worker and data tiers sit behind it on a private network.

The containers, and why each exists

Container	Runs	Blue-green?	Why
<code>app-web-blue</code> / <code>app-web-green</code>	NestJS API + baked-in React dashboard (<code>ROLE=web</code>)	✅ two copies	User-facing; needs zero-downtime cutover. Idle copy = warm rollback.
<code>app-worker</code>	BullMQ processors + cron scheduler (<code>ROLE=worker</code>)	❌ single	Background work; must be a singleton (see §3).
<code>postgres</code>	PostgreSQL 18 (reporting database)	❌ shared	Single source of truth; both colours share it.
<code>redis</code>	Redis 8 (queues + sessions)	❌ shared	Shared job queue and session store.
<code>caddy</code>	Reverse proxy + TLS	❌	The traffic switch; owns HTTPS certificates.

KEY IDEA

Both web colours run continuously. Seeing `app-web-blue` and `app-web-green` as "Up" is normal — that is what makes rollback instant. The file `active.conf` is the single source of truth for which one is actually receiving traffic.

One Image, Two Roles

A single Docker image runs two different ways, chosen by a `ROLE` environment variable. This split is the load-bearing design decision that makes blue-green correct.

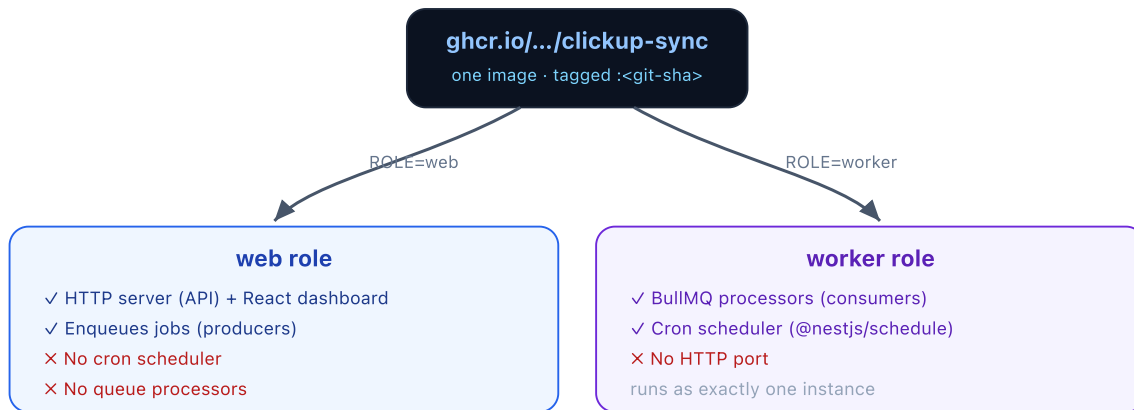


Figure 2 — The same artifact boots as a web server or a background worker depending on `ROLE`. Compose sets it per service.

Why the worker must be separate

Blue-green keeps the **old web copy alive during the whole rollback window**, not just for a moment. If the cron scheduler ran inside every web copy, then **two schedulers** would fire simultaneously — double-enqueuing backfills — and the old copy would process jobs with old code. Moving the scheduler and queue processors into a single dedicated `app-worker` eliminates that entire class of bug.

THIS REQUIRED AN APP-CODE CHANGE

"Dockerize + CI/CD" turned out to require a real NestJS change: `ScheduleModule` and the processor registrations are gated behind `ROLE` in `app.module.ts`, `sync.module.ts`, and `auth.module.ts`. Job *producers* stay in both roles; only *consumers* and the scheduler move to the worker. In the web role, `main.ts` boots the DI container without opening an HTTP port.

The Docker Image

A two-stage build compiles everything in a fat "build" stage, then copies only what's needed into the "runner" stage that actually ships.

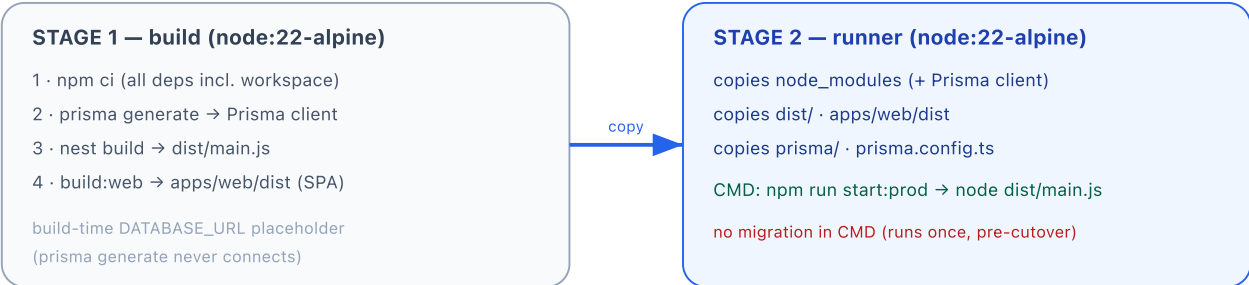


Figure 3 — Multi-stage Dockerfile. The dashboard is baked into the image, so the SPA and API always ship and version together.

The non-obvious decisions (and why they matter)

Decision	Reason
npm ci in the build stage	Provides apps/web workspace deps so build:web works without relying on hoisting.
prisma generate at build, client copied to runner	Otherwise the container boots with no Prisma client and crashes.
Full node_modules kept in runner	The Prisma CLI and prisma.config.ts loader are devDependencies, needed for migrate deploy .
Build-time DATABASE_URL placeholder	prisma.config.ts must resolve the var; generation never connects. The real URL is injected at runtime.
Flat dist/main.js output	start:prod , the COPY layout, and the SPA path all assume a flat build; a config fix pins rootDir so the compiler emits it flat.
Migration removed from CMD	With a shared DB and two colours, migrations must run once before cutover — not on every container boot (see §8).

The CI/CD Pipeline

Defined in `.github/workflows/deploy.yml` . Triggered on `push → main` (and manual `workflow_dispatch`). Four jobs run in a strict dependency chain.

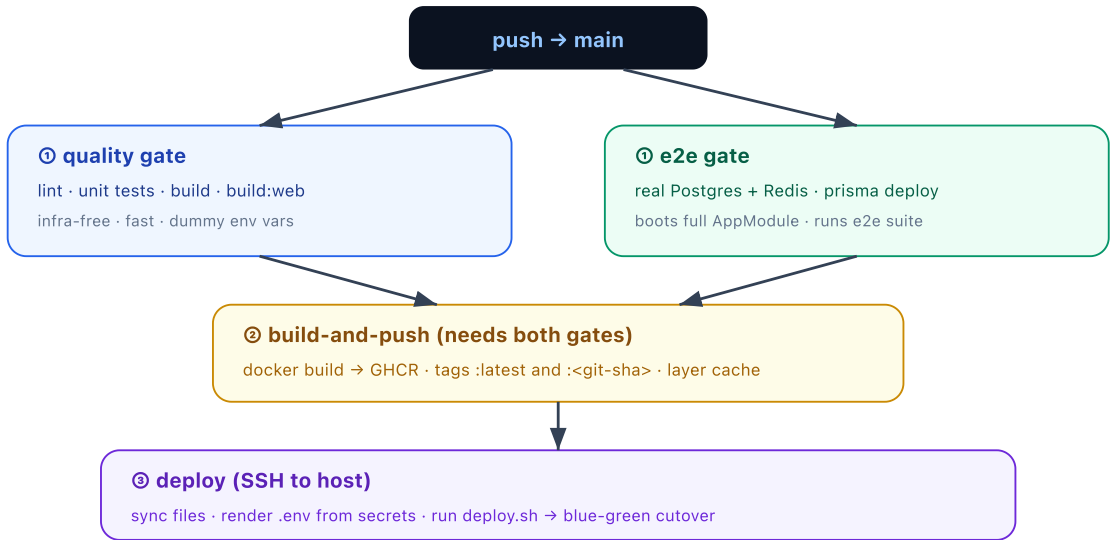


Figure 4 — The four-job pipeline. The two gates run in parallel; the build waits for both; deploy waits for the build. A failure anywhere stops the release.

Stage by stage

Job	What it does & why
quality	Runs <code>lint</code> , unit tests, <code>build</code> , and <code>build:web</code> with dummy env values. Kept infra-free so it's fast and can't be flaky.
e2e	Spins up real Postgres + Redis service containers, applies migrations, boots the full <code>AppModule</code> , and runs the end-to-end suite. This is the safety net that would catch a broken boot <i>before</i> any deploy.
build-and-push	Builds the image once and pushes it to GHCR tagged <code>:latest</code> and <code>:<git-sha></code> . The SHA tag is the immutable artifact the deploy pulls. Uses GitHub Actions layer caching.
deploy	Connects to the host over SSH, syncs <code>docker-compose.prod.yml</code> / <code>Caddyfile</code> / <code>scripts/deploy.sh</code> , renders <code>.env</code> from secrets, and runs the blue-green script.

CONCURRENCY GUARD

The workflow uses `concurrency: deploy-production` with `cancel-in-progress: false` — two deploys never run against the host at once; an in-flight deploy is allowed to finish.

The artifact

The image is the unit of deployment. It is built **once** per commit and reused everywhere: the same `:<git-sha>` image runs as web and as worker. Because the tag is the commit SHA, the running version is always traceable to exact source, and rolling back is just deploying a previous SHA that still lives in GHCR. Authentication to GHCR uses the ephemeral, job-scoped `GITHUB_TOKEN` — no long-lived registry secret is stored on the server.

Blue-Green Deployment

The heart of the system. A release upgrades the idle colour, proves it healthy in private, then flips traffic with a graceful reverse-proxy reload — never touching the copy users are on until the new one is confirmed good.

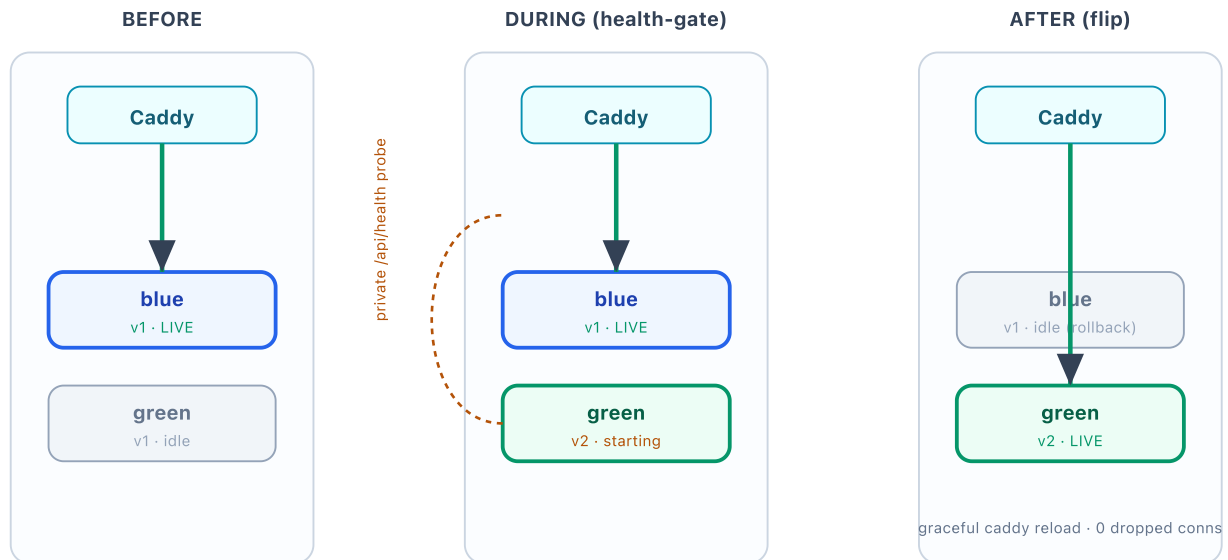


Figure 5 — The cutover. Green is upgraded and health-checked while blue still serves everyone. Only after green passes does Caddy flip; blue stays as the warm rollback target.

The cutover mechanism

Caddy's config is `{ $DOMAIN } { import /etc/caddy/active.conf }`. The file `active.conf` holds exactly one line — `reverse_proxy app-web-green:3000` (or `...-blue`). Flipping traffic is: rewrite that one line and run `caddy reload`, which is **graceful** — in-flight requests drain, zero connections dropped. That file is also the durable record of which colour is live, surviving reboots.

What `deploy.sh` does, in order

```
# scripts/deploy.sh — runs on the host, invoked over SSH by the deploy job
1. docker login ghcr.io # ephemeral GITHUB_TOKEN
2. docker compose pull # fetch image :<git-sha>
3. up -d postgres redis caddy # ensure infra is up
4. compose --profile tools run --rm migrate # ONE-SHOT migration, before any cutover
5. read active.conf → CURRENT ; TARGET = the other colour
6. up -d --force-recreate app-web-$TARGET # start idle colour on new image
7. HEALTH-GATE: poll TARGET /api/health (30 × 2s) via the caddy container
   └─ if never healthy → exit non-zero, DO NOT flip (old colour keeps serving)
8. rewrite active.conf → TARGET ; caddy reload # the flip (graceful)
9. up -d --force-recreate app-worker # update the singleton worker
10. docker image prune -af --until=168h # reclaim disk
```

THE SAFETY PROPERTY

Traffic only moves after the new colour passes its health check. If step 7 fails, the deploy exits with an error and the old colour is still live — a bad build can never take the site down. Migrations (step 4) always run **once, before** any cutover.

Rollback model

Situation	How to roll back
Just after a bad deploy (old colour still running)	Rewrite <code>active.conf</code> back to the previous colour + <code>caddy reload</code> . Instant , no rebuild.
Later (idle colour already replaced)	Re-run the deploy from the previous good commit / SHA — the image is still in GHCR.

Secrets Management

All configuration lives in **GitHub repository secrets**. The containers can't read GitHub secrets directly, so the deploy renders them into a `.env` file on the server at deploy time.

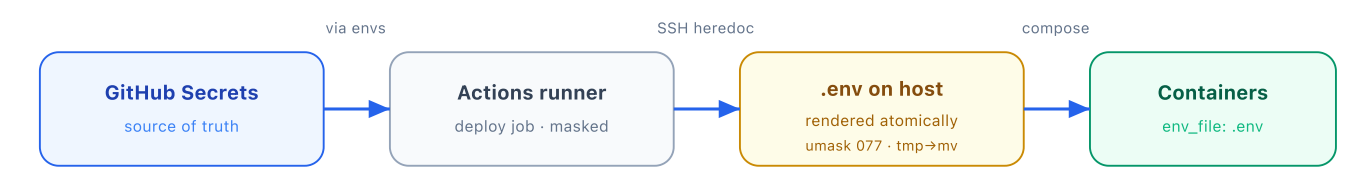


Figure 6 — Secrets flow. Values travel via the SSH action's `envs` list (so they're log-masked) and are written to `.env` with an atomic temp-file-then-move; no secret is ever echoed.

WHY EDITING `.ENV` ON THE SERVER DOESN'T STICK
Because CI **regenerates** `.env` from the secrets on every deploy, a manual edit to the file is overwritten. To change configuration permanently, update the **GitHub secret** and redeploy — never the file on the box.

The secrets, grouped

Group	Secrets
Deploy / SSH	SSH_HOST , SSH_USER , SSH_KEY , SSH_PORT , DEPLOY_PATH
Data	DATABASE_URL (host postgres), POSTGRES_PASSWORD , REDIS_URL (host redis)
App	APP_ENCRYPTION_KEY , ADMIN_API_KEY , DOMAIN , ALLOWED_ORIGINS , APP_BASE_URL
Email	SMTP_HOST , SMTP_PORT , SMTP_USER , SMTP_PASS , MAIL_FROM
Registry	none — GHCR uses the built-in GITHUB_TOKEN

ClickUp connection values are managed from the dashboard (stored encrypted in the database) and are therefore optional in the environment. Inside the Docker network, `DATABASE_URL` must use host `postgres` and `REDIS_URL` host `redis` — the compose service names, not `localhost` .

Database Migrations

Blue and green share one database, so migrations run **once** per deploy — before the cutover — and must be **backward-compatible**.

Run once, before cutover

Migrations were moved out of container startup into a dedicated one-shot `migrate` compose service (behind a `tools` profile so a normal `up` never starts it). The deploy runs `docker compose --profile tools run --rm migrate` at step 4, before any traffic moves. If it fails, the deploy stops before touching the live colour.

Forward-only / expand–contract (a permanent discipline)

Because the **old** colour stays alive during the rollback window, it must keep working against the **new** schema. Therefore every migration must be backward-compatible:

Phase	Rule
Expand	Add nullable columns, new tables, new indexes. Deploy code that tolerates both old and new shapes.
Contract	Only in a <i>later</i> deploy, once no running colour depends on the old shape, drop or rename.

NEVER

Drop or rename a column in the **same** deploy that introduces its replacement. That breaks the instant-rollback guarantee — a rollback flips *code*, it never un-migrates the *schema*.

Networking & TLS

Caddy terminates TLS and reverse-proxies to the live colour. The one genuinely fiddly part of the whole system is how TLS certificates interact with a Cloudflare proxy in front.

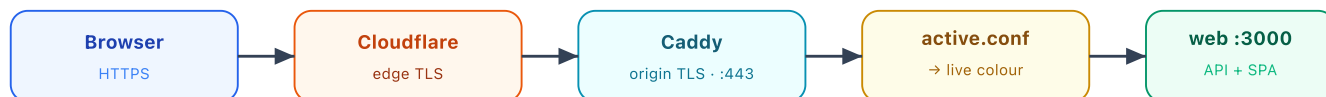


Figure 7 — Request path. Two TLS hops: browser→Cloudflare, and Cloudflare→Caddy. Both must succeed for the site to load.

The Cloudflare interaction

Caddy obtains its own certificate from Let's Encrypt automatically — but this requires the ACME challenge to reach Caddy. When a domain is **proxied through Cloudflare** (orange-cloud), Cloudflare intercepts the challenge and terminates TLS, so Caddy cannot complete validation and has no trusted certificate. Cloudflare then fails to handshake with the origin. Two symptoms seen in practice:

Symptom	Meaning & resolution
Error 525	Cloudflare reached the origin on 443 but TLS failed — Caddy has no valid cert for that hostname. Fix by giving Caddy a real cert (below).
ERR_TOO_MANY_REDIRECTS	Cloudflare SSL mode set to Flexible (talks HTTP to origin) while Caddy redirects to HTTPS — an infinite loop. Fix: set mode to Full (strict) .

Two supported resolutions

- **DNS-only (grey-cloud)**: disable the Cloudflare proxy for the record → Caddy completes ACME and serves a real Let's Encrypt certificate directly. Simplest; loses the CDN/proxy.
- **Keep proxy + Cloudflare Origin Certificate**: generate a long-lived Origin Certificate in Cloudflare, have Caddy serve it, and set Cloudflare SSL mode to **Full (strict)**. No public ACME needed — the production-grade way to keep Cloudflare in front.

ALSO REQUIRED ON CLOUD HOSTS

The instance firewall / security group must allow inbound **80** and **443** (and **22** for the CI SSH deploy). A blocked port 80 shows up as a **522** during ACME. Postgres/Redis ports are deliberately *not* opened.

Operations Runbook

Day-to-day commands. All run on the host from `/opt/clicksy` ; prefix with `sudo` unless the user is in the `docker` group.

Deploy

Push to `main` , or trigger manually: `gh workflow run deploy.yml` . Nothing else — the pipeline does the rest.

Which colour is live?

```
# the single source of truth:
cat /opt/clicksy/active.conf      # → reverse_proxy app-web-green:3000 ⇒ green is live
```

Status & logs

```
C="docker compose -f docker-compose.prod.yml"
$C ps                                # all containers + health
$C logs -f app-web-blue             # follow one service
$C logs --tail 100 app-worker      # last 100 lines
$C logs --since 15m -f caddy       # TLS / cert debugging
```

Confirm the live app is healthy

```
$C exec -T caddy wget -q0- \
  "http://$(grep -oE 'app-web-(blue|green)' active.conf):3000/api/health"
# → {"status":"ok","info":{"postgres":{"status":"up"},"redis":{"status":"up"}}}
```

Manual rollback (instant, if old colour still up)

```
printf 'reverse_proxy app-web-blue:3000\n' > active.conf # or -green
$C exec -T caddy caddy reload --config /etc/caddy/Caddyfile
```

Access the database

```
$C exec postgres psql -U clickup -d clickup_sync
# \dt list tables · \d clickup_tasks · \q quit
```

Postgres is not published to the host; for a GUI, tunnel through SSH to a temporary `socat` bridge rather than opening the port.

Troubleshooting — War Stories

Every real failure hit while bringing this online, what it looked like, the root cause, and the permanent fix. These are the most valuable pages for the next engineer.

① `npm ci` fails — `EUSAGE` lockfile out of sync

Cause: the committed `package-lock.json` drifted from the resolved dependency tree (transitive native-helper versions).

Fix: regenerate with `npm install` (package.json unchanged), commit the lockfile. CI's `npm ci` now passes. `commit 68d67ee`

② Container builds but won't boot — empty `dist/`

Cause: `nest build` emitted `dist/src/main.js` while everything assumed a flat `dist/main.js`; a stale incremental build-info file could even make the build skip emit entirely.

Fix: pin `rootDir: ./src` in the build tsconfig and disable incremental emit; ignore `*.tsbuildinfo` in the Docker context. `commits c746845, 68d67ee`

③ Web app crashes on boot — `SchedulerOrchestrator` dependency error

Cause: the ROLE split removed the scheduler from the web role, but two `@Cron` providers were still registered unconditionally, demanding a `SchedulerRegistry` that no longer existed.

Fix: gate the cron providers themselves by role in `sync.module.ts` and `auth.module.ts`. Caught before production by the e2e gate.

④ Deploy aborts at migration — "no such service: migrate"

Cause: the `migrate` service is behind a `tools` profile; `docker compose run migrate` can't find it without activating the profile.

Fix: `docker compose --profile tools run --rm migrate`. `commit aa6d112`

⑤ Postgres restart-loops — `unhealthy` on a fresh volume

Cause: Postgres 18 stores data in a version subdirectory and rejects a mount at `/var/lib/postgresql/data` ("unused mount/volume").

Fix: mount the volume at `/var/lib/postgresql` and add a health-check `start_period`. `commit 99b0968`

⑥ Health gate times out although the app is healthy

Cause: `deploy.sh` probed with `docker exec caddy ...`, but Compose names the container `<project>-caddy-1`, so `docker exec caddy` failed every time — the gate never saw the healthy app and never flipped.

Fix: probe via `docker compose exec -T caddy ...`, which resolves the service name regardless of project prefix. The blue-green safety worked correctly throughout — it simply refused to flip to what it couldn't verify. `commit 3d0f70a`

⑦ GHCR push rejected — wrong namespace

Cause: the image path pointed at a different account than the repo owner; `GITHUB_TOKEN` can only push to the owner's namespace.

Fix: align the image to `ghcr.io/<repo-owner>/clickup-sync`. `commit 1021ed4`

PATTERN WORTH NOTICING

In several of these, the pipeline's **guards did their job**: the e2e gate would have caught the boot crash, and the health gate refused to route traffic to a container it couldn't verify. The system failed *safe* every time.

Server Provisioning

The pipeline deploys to whatever host the secrets point at. Readyng a fresh Ubuntu box is a short, one-time checklist.

One-time host setup

```
# 1 · Docker Engine + Compose plugin
curl -fsSL https://get.docker.com | sudo sh
# 2 · run docker without sudo (re-login to apply)
sudo usermod -aG docker "$USER"
# 3 · deploy directory (must equal the DEPLOY_PATH secret)
sudo mkdir -p /opt/clicksy && sudo chown "$USER:$USER" /opt/clicksy
# 4 · dedicated CI deploy key (never share the private half)
ssh-keygen -t ed25519 -C gh-deploy -f ~/.ssh/gh_deploy -N ""
cat ~/.ssh/gh_deploy.pub >> ~/.ssh/authorized_keys
chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys
```

Then, in GitHub, update the server-specific secrets

Secret	Value
SSH_HOST	the host's public IP (use a static/Elastic IP so it doesn't change)
SSH_USER	the login user (e.g. <code>ubuntu</code> on EC2, not <code>clicksy</code>)
SSH_KEY	the new private key from step 4

Everything else (DB/Redis/SMTP/app secrets) is host-independent and unchanged. Open **22/80/443** in the firewall / security group, point DNS at the new IP, then `gh workflow run deploy.yml` . Postgres starts empty and all migrations re-run automatically.

Security Posture & Appendix

Security guarantees

- **Least exposure:** only Caddy publishes ports (80/443). Postgres and Redis are reachable only inside the Docker network.
- **No long-lived registry creds on the host:** GHCR auth uses the ephemeral, per-job `GITHUB_TOKEN` ; `docker logout` follows the pull.
- **Secrets never printed:** values move through the SSH action's `envs` list (log-masked) and are written with `umask 077` via an atomic temp-then-move; `.env` is git-ignored and never baked into an image layer.
- **Encrypted app settings:** ClickUp tokens and webhook secrets are stored encrypted in the database, protected by `APP_ENCRYPTION_KEY` (required in production).
- **TLS everywhere:** HTTPS at the edge and, in the recommended setup, encrypted Cloudflare→origin via Full (strict).

File reference

File	Purpose
<code>.github/workflows/deploy.yml</code>	The pipeline: gates → build → deploy.
<code>Dockerfile</code>	Multi-stage image (web + worker, baked-in SPA).
<code>docker-compose.prod.yml</code>	The stack: web x2, worker, migrate (profile), postgres, redis, caddy.
<code>scripts/deploy.sh</code>	Host-side blue-green cutover orchestration.
<code>Caddyfile</code> + <code>active.conf</code>	Reverse proxy + the live-colour switch.
<code>src/config/role.ts</code>	Resolves <code>ROLE</code> (web/worker).
<code>docs/OPERATIONS.md</code>	The canonical operations runbook.

Glossary

Term	Meaning
Blue-green	Two production copies; upgrade the idle one, health-check, then flip traffic.
Live / idle colour	Live = receiving traffic; idle = warm standby / rollback target.
Artifact	The SHA-tagged Docker image — the immutable unit of deployment.
Health gate	Polling <code>/api/health</code> on the new colour before flipping.
Expand/contract	Backward-compatible, two-phase schema change discipline.
GHCR	GitHub Container Registry — where images are published.
ACME	The protocol Caddy uses to obtain Let's Encrypt certificates.

